Objectifs de la feuille

- La vue générique de connexion
- La vue générique de d'inscription
- La vue générique de déconnexion
- Un peu de décoration

La vue générique de connexion

Il existe plusieurs manières d'utiliser le login/logout de base de DJANGO. Nous verrons dans cette partie pas forcément la manière la plus facile à mettre en œuvre, mais la plus souple. Commençons par la vue générique de connexion :

```
from django.contrib.auth import authenticate, login
from django.contrib.auth.views import LoginView

class ConnectView(LoginView):
	template_name = 'monApp/page_login.html'

	def post(self, request, **kwargs):
		identifiant= request.POST.get('username', False)
		motdepasse= request.POST.get('password', False)
		user = authenticate(username=identifiant, password=motdepasse)
		if user is not None and user.is_active:
			login(request, user)
			return render(request, 'monApp/page_home.html', {'param': identifiant, 'message': "You're connected"})
		else:
			return render(request, 'monApp/page_register.html')
```

Comme vous pouvez le constater, nous avons besoin du fichier page_login.html, le voici :

```
{% extends 'monApp/base.html' %}
{% block contenu %}
<div class="align-center col-8 offset-2">
	<p class="lead mb-3 ml-0">Se connecter</p>
	<hr/>
	<form method="POST" action="{% url 'login' %}" autocomplete="off">
	{% csrf_token %}
		<div class="form-group mb-3">
			<label>Identifiant</label>
			<input class="form-control" type="text" name="username" placeholder="Utilisateur">
			<label>Mot de Passe</label>
			<input class="form-control" type="password" name="password" placeholder="Mot de passe">
		</div>
		<button type="submit" class="btn btn-success mt-2">Se connecter</button>
	</form>
</div>
{% endblock %}
```

Quelques explications s'imposent : lorsque nous arrivons sur la vue, le template associé nous propose un formulaire où l'utilisateur devra saisir son identifiant et son password.

A la validation du formulaire (POST), la méthode post() de la classe ConnectView récupère les éléments du formulaire (username et password).

Si l'utilisateur est connu en BDD dans la table DJANGO auth.user à l'aide de la méthode authenticate(), l'utilisateur est connecté avec la méthode login() et est redirigé sur la page de bienvenue.

La vue générique d'inscription

Sinon, il doit s'inscrire sur la page page_register.html dont voici le template :

```
{% extends 'monApp/base.html' %}

{% block contenu %}
<div class="align-center col-8 offset-2">
        <p class="lead mb-3 ml-0">S'inscrire</p>
        <hr/>
        <form method="POST" action="{% url 'register' %}" autocomplete='off'>
        {% csrf_token %}
                <div class="form-group mb-3">
                        <label>Identifiant</label>
                        <input class="form-control" type="text" name="username" placeholder="Utilisateur">
                        <label>Mail</label>
                        <input class="form-control" type="mail" name="mail" placeholder="Email">
                        <label>Mot de Passe</label>
                        <input class="form-control" type="password" name="password" placeholder="Mot de passe">
                </div>
                <button type="submit" class="btn btn-success mt-2">S'inscrire</button>
        </form>
</div>
{% endblock %}
```

Même principe, un formulaire, trois champs à saisir et un clic sur le bouton « S'inscrire » qui nous envoie, via la méthode POST du formulaire et l'url portée par son action, vers la vue RegisterView :

```
from django.contrib.auth.models import User

class RegisterView(TemplateView):
        template_name = 'monApp/page_register.html'

        def post(self, request, **kwargs):
                identifiant = request.POST.get('username', False)
                courriel = request.POST.get('mail', False)
                motdepasse = request.POST.get('password', False)
                user = User.objects.create_user(identifiant, courriel, motdepasse)
                user.save()
                if user is not None and user.is_active:
                        return render(request, 'monApp/page_login.html')
```

La méthode post() de la classe RegisterView récupère les éléments du formulaire (identifiant, mail et password). L'utilisateur est créé en BDD dans la table DJANGO auth.user à l'aide de la méthode User.objects.create_user(). Si la création se passe bien, l'utilisateur est redirigé vers la page login.html pour se connecter ou bien il reste sur place…

Pour que tout cela se passe à merveille, vous aurez besoin des nouveaux éléments suivants dans le fichier monApp/urls.py

```
path('login/', views.ConnectView.as_view(), name='login'),
path('register/', views.RegisterView.as_view(), name='register')
path('logout/', views.DisconnectView.as_view(), name='logout'),
```

La vue générique de déconnexion

Il nous reste la vue DisconnectView qui hérite de TemplateView à implémenter dans views.py :

```
class DisconnectView(TemplateView):
        template_name = 'monApp/page_logout.html'

        def get(self, request, **kwargs):
                logout(request)
                return render(request, self.template_name)
```

Et son template associé tout simple page_logout.html :

```
{% extends 'monApp/base.html' %}

{% block contenu %}
<div class="align-center col-8 offset-2">
	<p class="lead mb-3 ml-0">Se déconnecter</p>
	<hr/>
	<form method="POST" action="{% url 'home' %}" autocomplete="off">
		{% csrf_token %}
		<div class="form-group mb-3">
			<label>Vous avez été déconnecté.</label>
		</div>
	</form>
</div>
{% endblock %}
```

Enfin, mettons à jour notre menu dans le fichier base.html avec ces éléments de navigation conditionnés au fait que l'utilisateur est connecté ou non. A vous de mettre ces lignes au bon endroit :

```
{% if user.is_authenticated %}
	<li class="nav-item">
	<a class="nav-link" href="{% url 'logout' %}">Déconnexion</a>
	</li>
{%else%}
	<li class="nav-item">
	<a class="nav-link" href="{% url 'login' %}">Connexion</a>
	</li>
	<li class="nav-item">
	<a class="nav-link" href="{% url 'register' %}">Inscription</a>
	</li>
{%endif%}
```

Au passage, vous aurez mis en œuvre deux formulaires, un peu perdus dans la nature au beau milieu de nos templates.

C'est à vous de jouer !

Vous venez de créer trois nouvelles vues... Ajustez vos tests (urls, contenus et validation de formulaire) pour chacune des vues afin de maintenir le taux de couverture.

Un peu de décoration

Oh non, encore du Bootstrap...Non non calmez-vous ! Nous allons parler décorateur au sens DJANGO. Le décorateur @login_required est utilisé pour restreindre l'accès à certaines vues aux utilisateurs authentifiés dans DJANGO. Voici comment l'utiliser correctement sur une fonction (vue basée sur une fonction) ou sur une classe (vue basée sur une classe).

Lorsqu'il s'agit d'une vue basée sur une fonction (FBV, Function-Based View), le décorateur @login_required est simplement appliqué au-dessus de la fonction :

```
from django.contrib.auth.decorators import login_required
from django.shortcuts import render

@login_required
def my_view(request):
        return render(request, 'my_template.html')
```

Ce décorateur vérifie si l'utilisateur est authentifié avant d'exécuter la vue. Si l'utilisateur n'est pas connecté, il est redirigé vers la page de connexion par défaut. Si vous souhaitez personnaliser l'URL de redirection pour les utilisateurs non authentifiés, vous pouvez ajouter l'argument login_url

```
@login_required(login_url='/monApp/login/')
def my_view(request):
        return render(request, 'my_template.html')
```

Pour les vues basées sur les classes (CBV, Class-Based View), ce qui est notre cas, vous ne pouvez pas utiliser directement le décorateur @login_required.
Cependant, DJANGO propose une méthode pour appliquer ce type de décorateur à des vues basées sur des classes en utilisant la fonction method_decorator( )

```
from django.contrib.auth.decorators import login_required
from django.utils.decorators import method_decorator
from django.views.generic import TemplateView

# Ajout du décorateur login_required à une CBV
@method_decorator(login_required, name='dispatch')
class MyView(TemplateView):
        template_name = 'my_template.html'
```

Ce décorateur @method_decorator(login_required, name='dispatch') applique @login_required à la méthode dispatch() de la classe, qui est responsable de la gestion des requêtes HTTP (GET, POST, etc.). Cela garantit que toutes les méthodes de la vue nécessitent que l'utilisateur soit connecté.

Si vous avez plusieurs méthodes dans la classe et que vous ne voulez appliquer @login_required qu'à certaines d'entre elles, vous pouvez aussi utiliser method_decorator à l'intérieur de la classe :

```
class MyView(TemplateView):
        template_name = 'my_template.html'

        @method_decorator(login_required)
        def get(self, request, *args, **kwargs):
                return super().get(request, *args, **kwargs)
```

Si vous souhaitez personnaliser l'URL de redirection pour les utilisateurs non authentifiés, vous pouvez ajouter l'argument LOGIN_URL = '/monApp/login/' dans le fichier monProj/settings.py.

C'est à vous de jouer !

Nous souhaitons que les actions de création, modification et suppression, des produits, des catégories, des statuts et des rayons soit soumis à l'authentification.
Mettez à jour vos tests sur les vues correspondantes car il faut que vous soyez maintenant connectés pour y accéder...